This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

Fast LM Head via Coarse-Fine Decomposition and Tile-Wise Weight Clustering

Jung-Woo Kim, Seung-Hwan Yoon, Hye-Rin Lee, Hak-Bum Lee, Su-Min Oh, Seung-Yoon Kang, Young-Ho Seo

***Abstract*—Large language models (LLMs) reuse each generated token as the next input during autoregressive decoding, making per-token latency a critical factor in practical inference performance. In this process, the LM Head performs a large-scale linear projection from the hidden state to the full vocabulary space at every decoding step, becoming one of the major bottlenecks in terms of memory bandwidth and latency. Existing output-layer approximation or vocabulary-pruning approaches can reduce the LM Head cost, but they may cause accuracy degradation due to logit-ranking distortion or task-dependent candidate omission. This paper reinterprets the LM Head as a candidate-screening problem and proposes a coarse-fine LM Head algorithm that restructures it into coarse and fine stages. The proposed method converts the LM Head weights into a compact codebook and an index map through tile-wise k-means clustering, and during online decoding, it computes full-vocabulary coarse scores using tile-wise LUTs and index streaming. Then, exact GEMV or GEMM using the original LM Head weights is performed only for the selected top-k candidates. Experimental results show that the proposed method limits the average accuracy change to -0.21%p compared with the original LM Head across nine downstream benchmarks, while accelerating the LM Head computation by 5.82× on average and up to 12.7×. In addition, end-to-end TPOT measurements demonstrate a speed-up of up to 1.18× on DGX Spark, and the method operates stably under various decoding lengths, batch sizes, and sampling conditions. The implementation and evaluation scripts are publicly available at https://github.com/HigherIdeal/CFHead.**

***Index Terms*—LLM, acceleration, LM Head, logits, GPU, bandwidth.**

I. Introduction

Since the advent of the Transformer architecture, large language models (LLMs) have achieved remarkable performance in natural language processing [1]. Most modern LLMs operate in an autoregressive manner, sequentially generating the next token by feeding the token generated at the previous step back into the input context [2], [3], [4]. In this process, at every decoding step, the final hidden state is projected through the LM Head onto the full vocabulary space, and the next token is selected based on the resulting logits. However, as the hidden dimension and vocabulary size of LLMs increase together, the LM Head is no longer merely a simple output layer [5], [6], [7], [8]. Instead, it becomes a large-scale vocabulary projection that must repeatedly access hundreds of megabytes to several gigabytes of weights for every generated token. In particular, because tokens are generated sequentially during decoding, the full-vocabulary GEMV of the LM Head can become a practical bottleneck in terms of latency and memory bandwidth.

Recently, studies have been proposed to reduce the burden of the LM Head by directly modifying output-layer computation or by reducing the vocabulary range that the LM Head must process during decoding [9], [10], [11]. SVD-Softmax [9] approximates the LM Head weights with a low-rank structure, thereby reducing the computational cost of full-vocabulary projection. This approach can be effective in approximating full-vocabulary logits at low cost, but logit distortion may occur due to low-rank approximation, and a sufficient rank is required to stably preserve the final token ranking.

With the emergence of small language models (SLMs), various methods have also been proposed to accelerate LLM decoding through speculative decoding using a draft model [12], [13], [14], [15]. Representative methods such as VocabTrim [12] and FR-Spec [15] reduce the computational range of the drafter LM Head in speculative decoding. VocabTrim reduces the drafter vocabulary based on token statistics frequently generated by the target model, while FR-Spec reduces the computational burden of the drafter LM Head by limiting the draft search range using a frequency-prioritized vocabulary subset. As the size of the original target model increases, speculative decoding can be highly effective in that a small draft model first generates multiple tokens and the target model verifies them [13], [14]. However, methods that rely on frequency-based vocabulary subsets, such as VocabTrim and FR-Spec, may fail to sufficiently include the target model's preferred tokens in situations that require long-tail vocabulary, such as multilingual generation, domain-specific documents, or creative generation. In this case, even if the final output can be corrected through verification by the target model, rejected draft tokens may increase, thereby reducing the acceptance rate and practical speed-up of speculative decoding.

In contrast, some studies modify the LM Head itself without relying on speculative decoding. FlashHead [11] assigns vocabulary token rows to balanced clusters based on the full hidden dimension, selects only a subset of clusters using the similarity between the hidden vector and the cluster centroids, and performs LM Head projection only for the token rows included in the selected clusters. However, because the candidates are determined not at the token level but as a set of selected clusters, it is difficult to directly control the number of fine-stage candidates, and increasing the number of probes to reduce cluster misses also increases the range of exact projection.

In this paper, we reinterpret LM Head logit generation not merely as a full-vocabulary projection but as a candidate-screening problem that selects promising vocabulary from the current hidden vector. Based on this perspective, we propose

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

a coarse-fine LM Head decomposition that first computes token-level coarse scores over the entire vocabulary and then performs exact projection only for the selected candidate tokens. To this end, the LM Head weights are divided into hidden-dimension tiles, and the vocabulary-wise weights of each tile are converted into a compact codebook and an index map through offline clustering. During decoding, an activation-dependent LUT is generated, and coarse scores are computed by streaming the index map and accumulating the corresponding LUT values. As a result, the coarse stage is performed by LUT lookups and reduction summation rather than full-precision GEMV, while the fine stage uses the original LM Head weights only for the selected candidate tokens, reducing both full-vocabulary weight access and the range of exact GEMV. Since the proposed method directly computes coarse scores for each token in the entire vocabulary rather than at the cluster level, the candidate set can be controlled at the token level. Therefore, compared with structures such as FlashHead, which must process all tokens within the selected clusters together, the range of fine-stage computation can be more explicitly limited. In addition, because the proposed method does not rely on a frequency-based vocabulary subset and instead uses the entire vocabulary as the target for coarse screening, it can alleviate candidate omissions caused by static subset pruning, even when long-tail tokens are required. Furthermore, in the coarse stage, after the activation-dependent LUT is generated, vocabulary-wise score accumulation is performed only with LUT lookups and additions, without multiplication. Therefore, this operation can be implemented as a simple reduction structure based on an adder tree and used to reduce memory bandwidth and arithmetic cost in accelerator or NPU design. Accordingly, the main contributions of this paper are as follows.

- Reformulation of LM Head logit generation from full-vocabulary projection into a candidate screening problem.
- A coarse-fine LM Head decomposition consisting of token-level coarse scoring and candidate-wise exact projection.
- Substantial elimination of multiplications in coarse score computation through a compact codebook and an index map.

The remainder of this paper is organized as follows. Section II introduces the computational structure of the LM Head and related prior studies. Section III describes the proposed coarse-fine LM Head algorithm and the tile-wise codebook-based coarse scoring method. Section IV evaluates the accuracy and efficiency of the proposed algorithm across various LLMs and benchmarks. Finally, Section V concludes this paper.

II. Related Works

In this section, Section II-A explains the computational structure of the LM Head and the token selection policies used in LLM decoding. Section II-B then reviews prior studies that reduce LM Head-centered output-side decoding cost, including approximation-based methods, speculative-decoding-based methods, and retrieval-based LM Head acceleration.

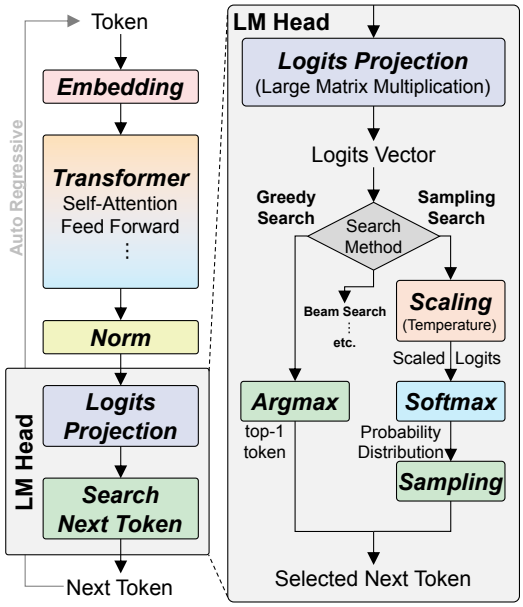


Fig. 1. LM Head computation structure in an LLM.

A. LM Head

LLMs based on the Transformer architecture generate text in an autoregressive manner by predicting the next token given the context [1]. At each decoding step, the final hidden state, which has passed through multiple Transformer Blocks, is fed to the LM Head, the final output layer. The LM Head is generally implemented as a very large linear projection layer and a token selection stage, and it projects the final hidden state into the vocabulary space to generate logits, which are unnormalized scores for each vocabulary token. Fig. 1 shows this LM Head-centered decoding flow, in which the final hidden state is converted into logits through the LM Head, and the next token is then determined according to the token selection policy.

The generated logits are used to select the next token according to the decoding policy. The simplest method is greedy search, in which the token with the largest value in the logits vector is selected as the next token. In contrast, probabilistic sampling first converts the logits into a probability distribution and then samples the next token from that distribution. In this process, temperature controls the sharpness of the distribution: a lower temperature yields outputs closer to those of greedy search, whereas a higher temperature increases the likelihood that more diverse tokens are selected. In practical decoding, top-k and top-p sampling are also used to restrict the candidate token set, and logit adjustments, such as repetition penalties, are applied to mitigate repetitive generation [16], [17], [18].

Meanwhile, changes in decoding policy do not always directly affect task accuracy. A prior study analyzed the problem-solving performance of LLMs by sweeping the sampling temperature from 0.0 to 1.6 and reported that, in particular, the performance difference was not statistically significant in the range of 0.0–1.0 [19]. Therefore, this paper focuses not on a specific token selection policy itself, but on the LM Head

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

structure that efficiently generates vocabulary-level logits or candidate token sets, which serve as inputs to various decoding policies.

B. LM Head Optimization

The LM Head can be viewed as a decoding-side computation stage that generates vocabulary-level scores from the final hidden state and selects the next token based on these scores. In LLMs with large vocabulary sizes, full-vocabulary projection and candidate search or sampling operations are repeated at every decoding step, making this process an important optimization target for computation and memory access. Prior studies for reducing LM Head-centered decoding cost can be broadly categorized into three directions. The first direction approximates output projection or softmax computation. The second reduces the total number of decoding steps or the output-side overhead of an auxiliary head through speculative decoding. The third reformulates the LM Head as a retrieval or candidate-search problem rather than a dense, full-vocabulary projection.

First, approximation-based methods use low-rank approximation, hierarchical structures, screening networks, or lookup-table substitution to reduce the cost of directly computing logits or softmax over a large vocabulary. SVD-Softmax aims to reduce inference cost by approximating large-vocabulary softmax computation based on singular value decomposition [9]. Learning to Screen first estimates a candidate word set likely to be in the current context using a lightweight screener, and then performs softmax only over that subset [10]. These methods can reduce output-side computation, but their application is often coupled to the model architecture or training procedure because they rely on low-rank factorization or a separate screening model.

The second direction aims to reduce the burden of the large-vocabulary LM Head during speculative decoding. Speculative decoding accelerates autoregressive decoding by allowing a draft model or a shallow draft path to generate multiple candidate tokens, which the target model then verifies in parallel. However, in large-vocabulary LLMs, the LM Head must compute scores over the entire vocabulary even in the draft stage. Therefore, even if the draft path is shallow, the cost of the output layer can limit the overall speed-up. To mitigate this issue, FR-Spec limits the draft search space using a frequency-ranked token subset and reduces the draft LM Head overhead during speculative sampling [15]. VocabTrim also proposed a method that reduces drafter-side output overhead by pruning the vocabulary of the drafter LM Head in speculative decoding [12]. However, because these methods limit the vocabulary search space during the draft stage, rejection rates can increase when the pruning or subset selection strategy is not sufficiently aligned with the target model's generation behavior. Therefore, the final speed-up is affected by draft quality, acceptance rate, and target-drafter agreement.

Beyond the conventional small-draft and large-target paradigm, speculative decoding has recently diversified into several emerging directions. Self-speculative decoding methods such as Draft & Verify [20], SWIFT [21], and KNN-SSD [22] construct the drafter from the target model itself via layer skipping or dynamic layer-set selection, and hidden-state-based methods such as EAGLE-3 [23], KV-reuse-based approaches [24], and retrieval-based hybrid tree construction [25] rely on different mechanisms for candidate generation and verification. With the rapid development of large vision-language models (LVLMs), speculative decoding has also been extended to multimodal settings [26], where methods such as SpecVLM [27] and LVSpec [28] combine the large-small speculative paradigm with visual-token pruning or visual-semantic guidance. Although these paradigms differ in how draft candidates are produced, the final token selection still involves scoring over the vocabulary through the LM Head in most of these pipelines. In other words, wherever a full-vocabulary output projection remains, the output-layer cost persists as a bottleneck independent of the drafting mechanism, and LM Head-side vocabulary reduction can be combined with these approaches in an orthogonal and complementary manner. The expected benefit is greatest when a draft-side LM Head is invoked repeatedly, whereas it may be smaller when candidate tokens are supplied directly through retrieval or when target verification amortizes the projection cost through batched computation. Accordingly, vocabulary reduction should be applied to reduce the output-layer cost without significantly degrading candidate coverage or acceptance rate. In LVLMs, this optimization complements rather than replaces visual-token reduction.

Recently, methods that reinterpret the LM Head itself as a retrieval problem rather than a dense classification head have also been proposed. FlashHead targets a training-free and hardware-friendly drop-in replacement, and reduces the cost of full-vocabulary dense projection by combining balanced clustering, multiprobe retrieval, inference-time sampling, and selective quantization [11]. This method is distinguished from speculative decoding approaches in that it directly targets the LM Head. However, in retrieval-based structures, latency and accuracy can vary with clustering quality, probing strategy, quantization precision, and sampling procedure. In particular, in structures where the cluster to which a token belongs, or the set of probed clusters, is restricted, tokens that could have achieved sufficiently high scores under the original LM Head scoring may be masked by the selected cluster set and excluded from the retrieval stage.

In summary, prior studies have attempted to reduce output-side decoding cost through softmax approximation, speculative decoding, vocabulary pruning, and retrieval-based search. However, softmax approximation methods can vary in performance and efficiency depending on the approximation structure and rank selection, while speculative decoding methods depend on draft quality and acceptance rate. Moreover, emerging speculative decoding paradigms, including self-speculative, hidden-state-based, and LVLM-oriented methods, incur the same output-side cost wherever a full-vocabulary output projection remains, indicating that LM Head-level optimization remains relevant across these settings. The practical gain nevertheless depends on how frequently and in what form the LM Head is invoked, and on whether candidate coverage and acceptance rate are preserved. FlashHead replaces the LM Head with a retrieval-based head, but there is a concern that

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

some tokens may be excluded from the candidate search range depending on the selected cluster set. Therefore, this paper proposes a method that addresses the concerns raised by prior studies and efficiently restructures the LM Head computation.

III. Proposed Algorithm

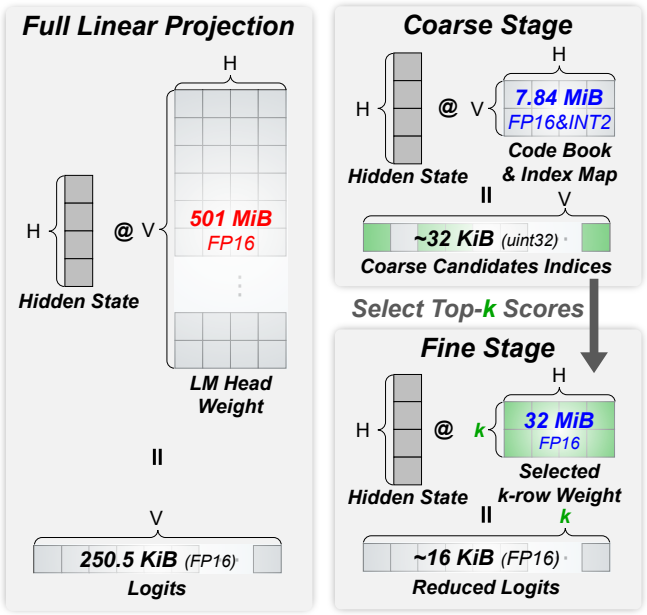
In this section, we describe a method for efficiently accelerating the LM Head. Section III-A describes a structure that decomposes the LM Head computation, and Section III-B describes a method that applies clustering to further improve the efficiency of the proposed method.

A. Coarse-Fine LM Head Approximation

As described in the previous section, the LM Head is one of the stages where the largest matrix multiplication occurs within the model. Given a hidden state with dimension H, the LM Head generates logits corresponding to the vocabulary size V through multiplication with a weight matrix of size $H \times V$. During decoding, only one token is generally generated at a time, so the entire $H \times V$ weight must be read from external memory at each step, which can be a major burden on decoding latency and memory bandwidth.

The simplest acceleration method is to quantize the entire LM Head weight to low-bit integers, as in existing precision-reduction studies, thereby enabling more weights to be transferred with the same memory bandwidth. However, because the LM Head performs classification over a very large vocabulary, if the entire LM Head weight matrix is aggressively compressed and logits are generated only from the compressed weights, even small quantization errors can cause changes in token rankings. That is, for low-frequency tokens or semantically similar candidates, even a small quantization error can change the ranking, so quantizing the entire LM Head weight to a very low bit-width, e.g., 1 bit, and generating logits only from it is not a safe approach.

To overcome these limitations and address this problem, we redefine the LM Head not as a single-stage matrix multiplication but as a two-stage structure comprising a coarse stage and a fine stage. First, a very lightweight, coarse LM Head quickly computes approximate scores for the entire vocabulary and extracts only the top-k candidate tokens. In the fine stage, full-precision LM Head computation is performed only for the columns corresponding to these top-k candidates, using the vectors from this top-k candidate set to determine the final argmax token. In other words, instead of performing one large high-precision matrix multiplication for all tokens, the structure is changed so that a lightweight approximate computation is performed for all tokens, and a precise matrix multiplication is performed only for a small set of candidates. That is, after rapidly generating coarse logits with high scores, the logits of the candidate set expected to belong to the top-k are accurately computed in the fine stage by loading only a subset of vectors from the original LM Head weights, thereby reducing the risk of ranking misses. This partly complements the problems that may arise from simple weight quantization alone. Fig. 2a illustrates how this LM Head computation method is changed.



(a) (b)

Fig. 2. Comparison of linear projection in LM Head between the original (a) and the proposed coarse-fine (b).

In the existing algorithm shown in Fig. 2a, given a hidden state with dimension H, the entire full-precision LM Head weight matrix of size $H \times V$ is read from DRAM at every decoding step. A GEMV operation then produces the full-vocabulary logit vector, from which the next token is selected using argmax for greedy decoding or a softmax-based sampling policy. Since all V columns must always be supplied from DRAM whenever the decoding step increases by one, the external memory bandwidth requirement and the amount of computation increase in direct proportion to the number of tokens.

In the proposed algorithm shown in Fig. 2b, coarse logits are first computed for the same hidden state using a compressed LM Head weight matrix. The coarse-stage weights are conceptually represented in an aggressively compressed form, such as a 1-bit or equivalent lightweight representation. In the proposed implementation, this representation is constructed through tile-wise vector clustering using the codebook and index map described in Section III-B. In particular, even when the codebook is stored in full precision, its size is only a few KiB, so it can reside in internal SRAM or be transferred with very low memory bandwidth. The computation structure can also be implemented more like a lookup of precomputed LUT values and tile-wise addition rather than a multiplication-centered GEMV, allowing coarse scores to be computed quickly for the entire vocabulary.

Meanwhile, it is desirable for the vocabulary-space projection results obtained using the reduced weights to form coarse candidate sets that sufficiently contain the original top-k tokens. However, simple quantization alone may fail to include tokens with high scores in the candidate sets passed to the fine stage, due to ranking misses in the coarse scores.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

This issue can be addressed by increasing the size of the coarse candidate sets, that is, by increasing top-k. However, this approach can degrade algorithmic efficiency by increasing the number of exact GEMV operations during the fine stage. Therefore, this paper uses tile-wise vector quantization and clustering to improve accuracy while reducing top-k, namely the size of the coarse candidate sets. Section III-B describes a method for effectively reducing the top-k sets extracted in the coarse stage while maintaining the accuracy of the LLM.

B. Tile-wise Vector Quantization with Clustering

As described in Section III-A, the core objective of the proposed coarse-fine LM Head is to construct a promising candidate token set in the coarse stage with low memory transactions and low computational cost, and then compute refined candidate logits in the fine stage by performing the original FP16 LM Head projection only for those candidates. These refined logits can then be used in greedy search or sampling-based decoding policies. Therefore, the coarse stage is designed not to directly determine a specific token, but as a screening stage that reliably includes tokens likely to have high scores in the original LM Head within a small candidate set. If the ranking of the coarse logits deviates significantly from the ranking of the original logits, important tokens may be omitted from the candidate set. If top-k is set excessively large to prevent this omission, the weight access of the fine-stage GEMV and the candidate selection cost increase, weakening the memory-traffic reduction effect of the coarse-fine structure. Therefore, the coarse stage should be designed not merely by reducing the bit-width of the weights, but by preserving the local distribution of the original LM Head weights within a limited storage cost.

To this end, this paper proposes a vector quantization method that clusters the LM Head weights offline on a tile basis. The proposed method converts the original LM Head weights into a small FP16 codebook and a compact index map, and this offline preprocessing procedure is shown in Fig. 3. First, the original LM Head weight matrix is partitioned along the hidden dimension into units of a fixed tile size. Each tile is a weight block that preserves the full vocabulary direction while containing only a partial interval of the hidden dimension. Therefore, each vocabulary token is represented by one local weight vector within each tile. Then, for each tile, the local weight vectors along the vocabulary direction are collected, and k-means clustering is independently performed on this set of vectors. As a result, centroid vectors representing the weight distribution of each tile are generated, and the set of these centroid vectors becomes the codebook used in the coarse stage. After clustering, an index map is generated to indicate which centroid each vocabulary row is closest to in each tile. That is, instead of storing the original weight values directly, only the centroid index is stored for each combination of vocabulary token and tile.

In the online decoding stage, coarse logits can be computed by looking up and accumulating the centroid contribution of each tile according to this index map. Therefore, in the coarse stage, approximate logits are generated using only the codebook and the index map, without directly reading the entire original LM Head weight matrix. Unlike simple scalar quantization or binarization, this structure preserves the local vector structure within each tile at the centroid level, and thus can more stably reflect the ranking of high-scoring tokens in the original logits.

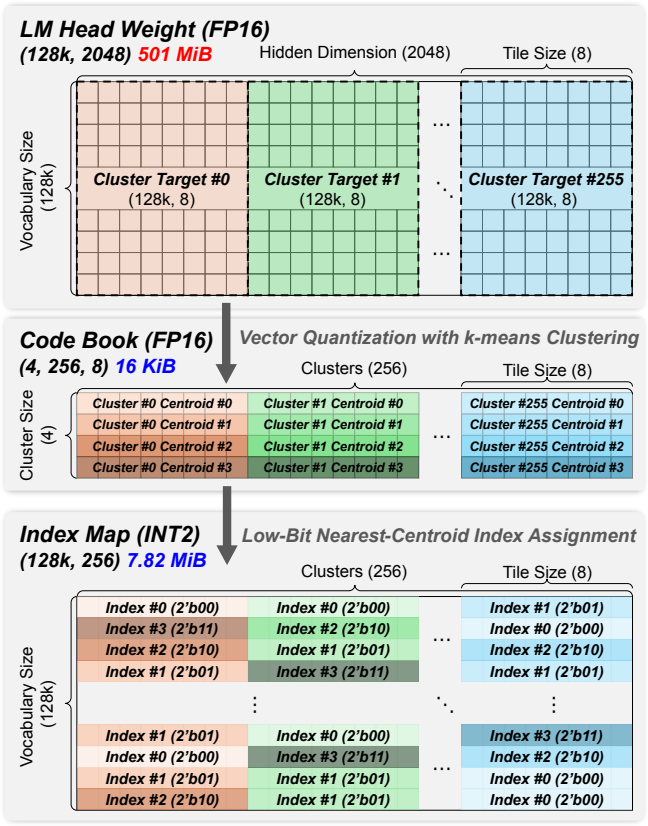


Fig. 3. Offline codebook and index-map construction through tile-wise clustering of the original LM Head weight in LLaMA3.2-1B.

The example in Fig. 3 is constructed based on LLaMA3.2-1B. The LM Head weight of this model has a vocabulary size of 128,256, a hidden dimension of 2,048, and a total size of approximately 501 MiB when stored in FP16 format. In Fig. 3, the tile size is set to 8, so the hidden dimension of 2,048 is divided into a total of 256 tiles. Therefore, each tile becomes a weight block of size 128,256 $\times$ 8, and k-means clustering is performed in each tile on 128,256 eight-dimensional weight vectors. In this example, the cluster size is set to 4. Therefore, four centroid vectors are generated for each tile, and the entire codebook consists of 4 $\times$ 256 $\times$ 8 FP16 values. This corresponds to approximately 16 KiB. In addition, since the cluster size is 4, each centroid index is represented by 2 bits. The index map consists of 128,256 $\times$ 256 INT2 entries, with a total size of approximately 7.82 MiB. As a result, the original 501 MiB FP16 LM Head weight is converted into an approximately 16 KiB codebook and an approximately 7.82 MiB index map for the coarse stage. This corresponds to approximately 63.9$\times$ storage reduction. This offline preprocessing is performed only once for the trained LM Head weight, and the generated codebook and index map

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

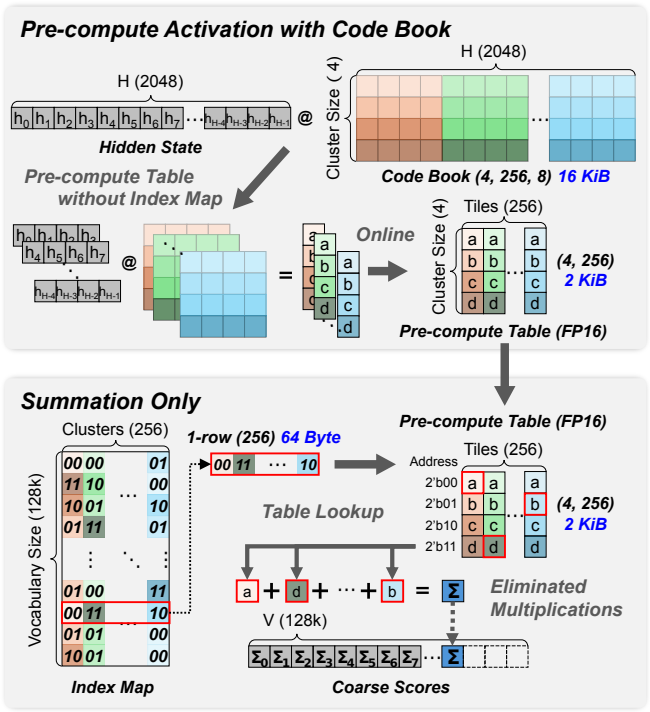


Fig. 4. Optimized coarse-stage computation using an online-generated pre-computed table for a batch size of 1.

are repeatedly reused at every decoding step. Therefore, the proposed method significantly reduces full-precision weight traffic in the coarse stage while preserving the vector-level weight distribution of each tile, increasing the likelihood that important tokens are included in a small candidate set.

Using the converted codebook and index map, a lookup table can be constructed online on-chip through a small amount of matrix multiplication between the hidden state and the codebook. Once this precomputed table is constructed with only a small amount of matrix multiplication, a very small table can eliminate a large number of multiplications in the coarse stage and replace them with element-wise reduction summation over the table entries. This process is shown in Fig. 4.

Figure 4 shows how the codebook and index map generated offline are used in the actual decoding stage. First, the hidden state output from the transformer is partitioned using the same tile size as that used in offline preprocessing. In this example, as in Fig. 3, the hidden dimension is 2,048 and the tile size is 8. Therefore, the hidden state is divided into 256 activation tiles. Each activation tile performs inner products with the centroid vectors of the codebook corresponding to that tile, and partial scores equal to the cluster size are computed for each tile. In this example, the cluster size is also 4, so four scores are generated for each tile. By collecting the results computed for the 256 tiles, a precomputed table of size 4×256 is constructed, corresponding to the tile-wise LUT referenced in the coarse stage. Since this table is only about 2 KiB in FP16 format, it can be sufficiently maintained in on-chip memory.

Afterward, coarse score computation is no longer performed using a full-vocabulary GEMV with the original LM Head weights. Instead, for each vocabulary row, the centroid index of each tile is sequentially read from the index map, and the corresponding value is looked up in the precomputed table using this index as an address. For example, one vocabulary row has a 2-bit index for each of the 256 tiles, and in this example, only 256×2 bits, i.e., 64 bytes, are required to represent one row. By accumulating all LUT entries pointed to by these 256 indices along the tile axis, the coarse score for each vocabulary token is obtained. Repeating this operation over the entire vocabulary constructs the full coarse logits.

That is, the key point of Fig. 4 is that most operations in the coarse stage are performed not by large-scale multiplication, but only by index streaming, LUT lookup, and reduction summation after a small amount of pre-computation. Actual multiplication is used only in a limited manner when generating the precomputed table from the hidden state and the codebook, and subsequent score generation over the entire vocabulary dimension is replaced by lightweight addition-centered computation. Therefore, in the coarse stage, it is not necessary to repeatedly read the entire original FP16 LM Head weights, and full-vocabulary coarse scores can be efficiently generated using only a small codebook and a compact index map. This structure reduces both memory traffic and computation in the coarse stage, while enabling the ranking of the original logits to be approximated with relatively stable accuracy based on the local weight structure preserved through offline clustering. In addition, each token has an independent coarse score according to the index map, and only the top-k tokens selected from the token-level coarse logits are delivered to the fine stage, rather than cluster-level candidates. Therefore, the number of tokens inside a cluster is not directly tied to the size of the fine-stage GEMV.

Accordingly, the proposed tile-wise vector quantization does not merely compress the LM Head weights to a lower bit-width. Instead, it structures the weight distribution into tile-wise codebooks and index maps offline, and computes full-vocabulary coarse scores online in an addition-centered manner by generating activation-dependent LUTs. In particular, clusters are not used as units to directly expand the fine-stage candidates, but only as lookup addresses to compute the coarse score of each token. As a result, the coarse logits are always constructed at the vocabulary-token level, and only the top-k candidates selected from the token-level coarse scores are delivered to the fine stage, rather than all tokens belonging to a cluster. Therefore, the proposed method has the advantage of reducing memory traffic and multiplication cost in the coarse stage, while controlling the candidate size of the fine-stage GEMV independently of the cluster population.

IV. Experimental Evaluation

This section quantitatively evaluates the proposed coarse-fine LM Head. Table I summarizes the main experiments by task, together with the tile size, cluster size, and coarse-stage top-k used in each. Rather than adopting a single default configuration, these values are operating points selected for the objective of each experiment under accuracy-speed-storage trade-offs; in particular, since a smaller cluster size lowers

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

TABLE I
EXPERIMENTAL TASKS AND HYPERPARAMETER SETTINGS.

Task	Tile Size	Cluster Size	Top-k
Accuracy Verification	8	256	2 to 64
Baseline Comparison	8	4	8,192
Various Scenarios Eval.	4	64	256
Speed-up Eval.	8	64	256 to 2,048
Ablation	4 to 512	4 to 512	2 to 8,192

the representation granularity of the coarse logits, top-k is increased together to maintain candidate coverage.

Section IV-A evaluates accuracy and robustness across various datasets, sampling settings, and batch sizes, and compares the proposed method with existing LM Head acceleration techniques, corresponding to Accuracy Verification, Baseline Comparison, and Various Scenarios Evaluation in Table I. Section IV-B measures the isolated LM Head speed-up and end-to-end decoding speed-up on real GPUs (Speed-up Evaluation). Section IV-C performs ablation studies on tile size, cluster size, and top-k. Most experiments were conducted on NVIDIA DGX Spark, with additional evaluations on RTX 4090 and A100 to cover consumer-grade and server-grade GPU environments.

A. Accuracy and Robustness

This section evaluates how accurately the proposed method reproduces the results of the original LM Head. To this end, top-1 hit ratio is used in some experiments. The top-1 hit ratio is defined as the fraction of decoding steps in which the top-1 token selected by the original LM Head is included in the top-k candidate set selected by the coarse stage. If the original top-1 token is included in the candidate set at the coarse stage, the fine stage performs the full-precision LM Head computation on those candidates, allowing the final token in greedy decoding to be selected identically to the original. Therefore, the top-1 hit ratio is a metric that indicates how stably the coarse stage includes the original top-1 token in the candidate set.

First, to verify the effectiveness of the proposed clustering-based coarse stage, we measured the top-1 hit ratio over 256 decoding steps using LLaMA3.1-8B-Instruct. In this experiment, "How to learn LLM?" was used as the input prompt, and greedy decoding was performed. The tile size and cluster size of the proposed method were set to 8 and 256, respectively, and top-k was varied from 2 to 64. A large cluster size of 256 is used in Figs. 5 and 6 to minimize clustering approximation error and thus isolate the ranking-preservation capability of clustering itself, while the small top-k range corresponds to this argmax-oriented evaluation. As comparison targets, we used the Weight Only method, which quantizes only the LM Head weights into 1-bit signs, the Act. + Weight method, which quantizes both the hidden state and the LM Head weights into 1-bit signs, and the proposed tile-wise clustering-based method. Fig. 5 shows how the top-1 hit ratio of each method changes as the top-k candidate set size increases.

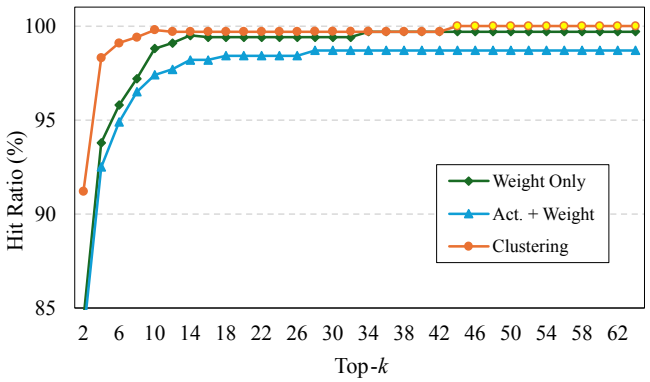


Fig. 5. Top-1 hit ratio versus top-k. Comparison of clustering-based coarse logits, weight-only binarization, and activation-and-weight binarization. Yellow markers indicate cases where the top-1 hit ratio reaches 100%.

In Fig. 5, the top-1 hit ratio increases with top-k for all three methods, but the rate at which it reaches 100% differs significantly. The Act. + Weight method, which restricts both activations and weights to 1-bit signs, yields the lowest ranking quality of the coarse logits and does not reach a 100% hit ratio even when top-k is increased to 64. The Weight Only method preserves the real-valued information of the activations and therefore shows a higher hit ratio, but it still fails to include the original top-1 token stably with only a small candidate set. In contrast, the proposed clustering-based method rapidly reaches a 100% hit ratio in a smaller top-k range, because tile-wise clustering represents the local vector patterns of the LM Head weights using a centroid codebook rather than simple sign-based quantization.

Next, we examined whether the proposed method can maintain the same level of accuracy as the original LM Head on downstream benchmarks. To this end, 5-shot prompts were constructed from 100 examples randomly sampled from the MMLU dataset, and the proposed method was applied to the LLaMA3 and Qwen2.5 families. In this experiment, the tile size and cluster size were also set to 8 and 256, respectively, and accuracy was measured as top-k was varied from 4 to 64. Fig. 6 shows the changes in MMLU accuracy according to the top-k candidate set size for both families, where the yellow markers indicate points at which the proposed method's accuracy matches the baseline accuracy of the original LM Head.

In Fig. 6, all models in both families reach the baseline accuracy as top-k increases; in particular, LLaMA3.2-1B, LLaMA3.1-8B, Qwen2.5-7B, and Qwen2.5-14B match the baseline even with relatively small candidate sets. These results indicate that the proposed method is not limited to a specific model family and can be consistently applied across different LLM families, such as LLaMA3 and Qwen2.5.

Therefore, the results in Figs. 5 and 6 show that the proposed clustering-based coarse stage includes the original top-1 token in the candidate set more effectively than simple 1-bit quantization, while achieving the same accuracy as the original baseline on downstream benchmarks. In the following experiments, we further evaluate the robustness of the pro-

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

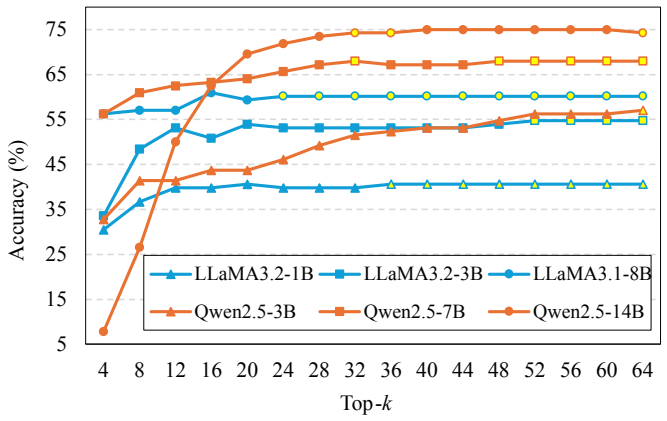


Fig. 6. Downstream accuracy versus top-k for LLaMA-family (LLaMA3.2-1B, LLaMA3.2-3B, LLaMA3.1-8B) and Qwen-family instruction models (Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-14B). Dashed lines denote the baseline accuracy of each model, and yellow markers indicate cases where accuracy matches the baseline accuracy.

TABLE II
TOKEN-LEVEL TOP-1 HIT RATIO UNDER LONG-FORM GREEDY DECODING.

Dataset	Steps	L3.2-1B	L3.1-8B
ShareGPT	8,192	100.0%	100.0%
	16,384	100.0%	100.0%
LMSYS-Chat-1M	8,192	100.0%	100.0%
	16,384	100.0%	100.0%
LongBench	8,192	100.0%	100.0%
	16,384	100.0%	100.0%

Note: L3.2-1B and L3.1-8B denote LLaMA3.2-1B and LLaMA3.1-8B, respectively.

posed method using longer decoding steps, various datasets, sampling-based decoding, code generation quality assessment, and comparison with existing LLM acceleration methods.

Table II reports the token-level top-1 hit ratio of the proposed method in long-form greedy decoding. This experiment uses the ShareGPT [29], LMSYS-Chat-1M [30], and LongBench [31] datasets, and the target models are set to LLaMA3.2-1B and LLaMA3.1-8B. The input sequence length is fixed to 256, and generation is performed up to 8,192 and 16,384 decoding steps using greedy decoding. The tile size and cluster size of the proposed method are set to 4 and 64, respectively, and the top-k of the coarse stage is set to 256. This configuration balances ranking fidelity against auxiliary cost, and a top-k of 256 serves as the reference operating value throughout the scenario evaluations in Tables II–V. In Table II, the proposed method achieves a token-level top-1 hit ratio of 100% for all combinations of dataset, model, and decoding length. This shows that the proposed method consistently includes the original top-1 token in the candidate set even during long greedy decoding of up to 16,384 steps, and that this behavior holds regardless of input characteristics, from diverse dialogue scenarios to long-context inputs.

However, practical LLM inference also widely uses probabilistic sampling, in which a token is selected from a distribution rather than as a single argmax. Accordingly, Tables III and IV measure, using ShareGPT, LMSYS-Chat-1M, and LongBench inputs, how often the token sampled from the original LM Head matches the token sampled from the proposed coarse-fine LM Head in terms of token-level hit ratio. Here, SG, LC, and LB denote ShareGPT, LMSYS-Chat-1M, and LongBench, respectively. Then, in Table V, to verify whether these token-level differences degrade actual generation quality, code generation performance is evaluated on the HumanEval+ benchmark using pass@1, pass@5, and pass@10.

TABLE III
TOKEN-LEVEL SAMPLING HIT RATIO OF THE PROPOSED METHOD UNDER NUCLEUS SAMPLING ON LIGHTWEIGHT MODELS.

Top-k	L3.2-1B			Q3.5-0.8B		
	SG	LC	LB	SG	LC	LB
32	86.5%	85.5%	87.0%	88.1%	90.8%	82.8%
64	88.6%	88.0%	90.4%	92.1%	93.8%	87.5%
128	90.2%	90.1%	92.8%	94.6%	95.9%	91.6%
256	91.4%	91.8%	94.6%	96.7%	97.3%	94.5%
512	92.4%	93.4%	96.1%	98.0%	98.4%	96.7%
1,024	93.7%	94.7%	97.3%	99.0%	99.0%	98.3%

Note: L3.2-1B, Q3.5-0.8B, denote LLaMA3.2-1B, Qwen3.5-0.8B respectively. SG, LC, and LB denote ShareGPT, LMSYS-Chat-1M, and LongBench, respectively.

TABLE IV
TOKEN-LEVEL SAMPLING HIT RATIO OF THE PROPOSED METHOD UNDER NUCLEUS SAMPLING ON MIDDLE-SCALE MODELS.

Top-k	L3.1-8B			Q3.5-9B		
	SG	LC	LB	SG	LC	LB
32	93.4%	93.2%	90.1%	96.2%	96.2%	88.8%
64	94.7%	94.8%	93.1%	97.5%	97.4%	92.6%
128	95.8%	96.2%	95.5%	98.5%	98.2%	95.0%
256	96.4%	97.1%	97.0%	99.0%	98.8%	96.9%
512	97.1%	98.1%	98.0%	99.3%	99.2%	98.2%
1,024	**97.5%**	**98.6%**	**98.8%**	**99.6%**	**99.5%**	**98.9%**

Note: L3.1-8B, Q3.5-9B, denote LLaMA3.1-8B, Qwen3.5-9B respectively. And SG, LC, and LB denote ShareGPT, LMSYS-Chat-1M, and LongBench, respectively.

Table III reports the token-level hit ratio for lightweight models under sampling-based decoding. This experiment uses inputs randomly selected from ShareGPT, LMSYS-Chat-1M, and LongBench, and is conducted on LLaMA3.2-1B and Qwen3.5-0.8B. The input sequence length is fixed to 256, and 256 decoding steps are generated for each sample. The average over 30 samples is used, and the decoding policy is set to nucleus sampling with a temperature of 1.0 and a top-p of 0.9. The tile size and cluster size of the proposed method are set to 4 and 64, respectively, and top-k is varied from 32 to 1,024. Unlike the greedy-oriented evaluation in Fig. 6, sampling requires covering multiple tokens within the top-p distribution rather than a single argmax token, so a wider top-k range is swept.

Table IV reports the results of performing the same sampling-based decoding experiment on middle-scale models, LLaMA3.1-8B and Qwen3.5-9B, with all other settings iden-

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

TABLE V
HUMANEVAL+ PASS@k COMPARISON BETWEEN THE ORIGINAL AND THE PROPOSED METHOD UNDER NUCLEUS SAMPLING WITH TEMPERATURE 0.6.

Metric	Setting	L-1B	L-8B	Q-0.8B	Q-9B
pass@1	Baseline	23.7%	54.9%	23.8%	80.9%
	Ours	23.7%	54.9%	23.8%	80.9%
	Δ	**0.0%p**	**0.0%p**	**0.0%p**	**0.0%p**
pass@5	Baseline	38.1%	68.0%	39.2%	90.7%
	Ours	38.1%	68.0%	39.4%	90.7%
	Δ	**0.0%p**	**0.0%p**	**+0.2%p**	**0.0%p**
pass@10	Baseline	44.5%	70.7%	45.7%	92.7%
	Ours	44.5%	70.7%	45.7%	92.7%
	Δ	**0.0%p**	**0.0%p**	**0.0%p**	**0.0%p**

Note: L-1B, L-8B, Q-0.8B, and Q-9B denote LLaMA3.2-1B, LLaMA3.1-8B, Qwen3.5-0.8B, and Qwen3.5-9B, respectively.

tical to those in Table III.

In Tables III and IV, the token-level hit ratio increases with top-k for all models and inputs. Even at a top-k of 32, the hit ratio exceeds 82% for the lightweight models and 88% for the middle-scale models, and at a top-k of 1,024 it reaches 93.7–97.3% for LLaMA3.2-1B and up to 99.6% for Qwen3.5-9B. Unlike greedy decoding, the hit ratio does not reach 100%, because the sampling process does not select a single argmax token, but probabilistically selects a token within the candidate distribution restricted by top-p. Nevertheless, high token-level hit ratios across both lightweight and middle-scale models indicate that the proposed method stably reflects the token selection tendency of the original LM Head even in sampling settings. However, a token-level mismatch does not directly imply degradation in generation quality: a token different from that of the original LM Head may still be valid in the context, and in code generation in particular, different token sequences may pass the same test cases. Accordingly, Table V evaluates the actual code generation quality in sampling settings using the HumanEval+ benchmark when the temperature is 0.6.

Table V compares the code generation performance of the original LM Head and the proposed coarse-fine LM Head on the HumanEval+ benchmark. This experiment was conducted on 164 Python programming problems, and 10 samples were generated for each problem under nucleus sampling at a temperature of 0.6. The input sequence length was set to 1,024. For the proposed method, top-k, tile size, and cluster size were set to 256, 4, and 64, respectively, and Δ denotes the difference between Ours and Baseline in %p. In Table V, pass@1, pass@5, and pass@10 are identical to the baseline for all models except Qwen3.5-0.8B, whose difference is at most +0.2%p. This shows that, under a condition where the stochasticity of sampling is relatively low, the coarse-fine LM Head almost fully preserves the code generation quality of the original LM Head.

Therefore, even if the coarse-fine LM Head does not always select the same token as the original LM Head in sampling decoding, the actual task-level generation quality can generally be maintained. Based on these results, Table VI further analyzes the relative performance of the proposed method by comparing it with existing LM Head acceleration methods.

Table VI compares the Original LM Head, VocabTrim, FlashHead, and the proposed method across nine downstream benchmarks using greedy search. Since all methods are compared as standalone LM Head replacements rather than as speculative pipelines, VocabTrim is applied as a static vocabulary-pruning baseline without target-model verification; its degradation on GSM8K and MGSM thus reflects the candidate-coverage limitation of a static frequency-based vocabulary, which in its original speculative setting would instead appear as increased draft rejections and a reduced acceptance rate (Section II-B). For the proposed method, a cluster size of 4 keeps the auxiliary memory comparable to the baselines, and a top-k of 8,192 ensures sufficient candidate coverage under this small cluster configuration. The Average column indicates the mean score over all benchmarks, and the value in parentheses denotes the difference from the Original baseline.

In Table VI, the proposed method records an average score of 46.46%, a decrease of only 0.21%p from the Original, whereas VocabTrim and FlashHead show average drops of 6.34%p and 10.3%p, respectively, with significant degradation concentrated on GSM8K and MGSM. This can be attributed to the possibility that each algorithm either reduces the vocabulary or fails to select truly promising tokens in the cluster probes used during inference. In contrast, the proposed method computes an independent coarse score for every token in the vocabulary, and achieves the best or second-best score on all benchmarks. This shows that the proposed method does not depend only on a specific benchmark, but can be used across various tasks and keeps accuracy degradation more limited than existing methods.

TABLE VI
ACCURACY COMPARISON ACROSS NINE DOWNSTREAM BENCHMARKS. AVERAGE VALUES INCLUDE DIFFERENCES FROM THE ORIGINAL BASELINE IN PARENTHESES. BOLD AND UNDERLINED VALUES DENOTE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY.

Method	Average	MMLU	ARC-Chal.	HellaS.	GSM8K	IFEval	TruthQA	PIQA	WinoG.	MGSM
Original	**46.67 (Baseline)**	**39.8%**	**46.9%**	37.5%	**40.6%**	48.9%	**75.0%**	57.8%	51.6%	**21.9%**
VocabTrim*	40.33 (-6.34%p)	**39.8%**	**46.9%**	37.5%	5.47%	41.1%	**75.0%**	57.8%	51.6%	7.81%
FlashHead	36.36 (-10.3%p)	31.3%	35.9%	37.5%	26.6%	**54.6%**	36.7%	47.7%	**52.3%**	4.69%
Ours	46.46 (-0.21%p)	**39.8%**	**46.9%**	**39.1%**	**40.6%**	49.4%	74.2%	**58.6%**	51.6%	18.0%

Note: ARC-Chal., HellaS., TruthQA, and WinoG. denote ARC Challenge, HellaSwag, TruthfulQA, and WinoGrande, respectively. Benchmarks: MMLU [32], ARC-Chal. [33], HellaS. [34], GSM8K [35], IFEval [36], TruthQA [37], PIQA [38], WinoG. [39], and MGSM [40]. * VocabTrim denotes a static vocabulary-pruning baseline applied directly to final generation without target-model verification.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

TABLE VII
MEMORY REQUIREMENT AND AVERAGE TPOT SPEED-UP FOR LOGITS COMPUTATION UNDER THE TABLE VI EVALUATION SETTING.

Method	Aux. Memory	Full Memory	TPOT Speed-up
Original	–	501 MiB	–
VocabTrim	–	32.0 MiB	1.47×
FlashHead	31.8 MiB	63.8 MiB	1.16×
Ours	7.84 MiB	39.8 MiB	1.18×

B. Runtime Performance

This section evaluates the runtime performance of the proposed coarse-fine LM Head from four perspectives: the memory footprint and average TPOT under the Table VI setting (Table VII), the isolated LM Head speed-up across various LLM families (Table VIII), the end-to-end decoding TPOT on RTX 4090, DGX Spark, and A100 (Table IX), and the batch-size sensitivity (Table X). For these runtime evaluations, a tile size of 8 is used to reduce the number of tiles and thus the LUT lookup and accumulation overhead, and a cluster size of 64 is chosen considering both coarse ranking quality and LUT construction cost; top-k is swept in Table VIII to expose the candidate-size trade-off and fixed to 256 in Tables IX and X.

Table VII compares the memory required for logits computation and the average TPOT speed-up under the same setting used for the benchmark comparison in Table VI. VocabTrim reduces the full memory from 501 MiB to 32.0 MiB through vocabulary pruning and shows the highest TPOT speed-up of 1.47×, but as shown in Table VI, this gain is accompanied by a 6.34%p average accuracy drop. Compared with FlashHead, the proposed method uses less auxiliary memory (7.84 vs. 31.8 MiB) and less full memory (39.8 vs. 63.8 MiB) while achieving a slightly higher TPOT speed-up (1.18× vs. 1.16×), and limits accuracy degradation to 0.21%p. Therefore, the proposed method is not merely a latency-reduction method, but has balanced characteristics among memory footprint, TPOT speed-up, and downstream accuracy.

Table VIII reports the isolated LM Head speed-up of the proposed method across various LLMs. This experiment was conducted on DGX Spark and targeted models from the LLaMA, Qwen2.5, Ministral/Mistral, OPT, and BLOOM families. Token selection was set to greedy decoding, and the speed-up over the original LM Head was calculated based on the average LM Head runtime. The tile size and cluster size of the proposed method were set to 8 and 64, respectively, and top-k was varied among 256, 512, 1,024, and 2,048.

In Table VIII, the proposed method accelerates LM Head computation across all model families, with an average speed-up of approximately 5.82×, a maximum of 10.7× (Qwen2.5-32B, top-k 256), and a minimum of 2.18× (OPT-350M, top-k 512). Overall, as top-k increases, the number of candidate weight vectors to be processed in the fine stage increases, so the speed-up tends to decrease, yet most models still maintain meaningful LM Head speed-up at a top-k of 2,048. This shows that the proposed method is not limited to a specific LLM family, but can effectively reduce LM Head computation across models with various vocabulary sizes and hidden dimensions.

TABLE VIII
LM HEAD-ONLY SPEED-UP OF THE PROPOSED METHOD ACROSS VARIOUS LLMS.

Model	Top-k			
	256	512	1,024	2,048
LLaMA3.2-1B	8.56×	8.23×	7.93×	7.03×
LLaMA3.2-3B	5.01×	5.06×	4.91×	4.78×
LLaMA3.1-8B	4.81×	4.93×	4.84×	4.63×
Qwen2.5-0.5B	7.71×	7.26×	7.41×	7.07×
Qwen2.5-1.5B	8.55×	8.31×	7.70×	7.01×
Qwen2.5-3B	6.63×	6.46×	6.02×	5.86×
Qwen2.5-7B	4.96×	4.90×	4.86×	4.63×
Qwen2.5-14B	5.54×	5.45×	5.36×	5.17×
Qwen2.5-32B	10.7×	10.3×	10.2×	9.80×
Ministral-3-3B	5.27×	5.23×	5.20×	4.95×
Ministral-3-8B	5.44×	5.41×	5.24×	5.12×
Mistral-Small-3.1-24B	5.23×	5.20×	5.08×	4.87×
OPT-125M	3.30×	3.15×	3.13×	2.87×
OPT-350M	2.44×	2.18×	2.30×	2.21×
OPT-1.3B	6.42×	6.11×	6.13×	5.63×
OPT-2.7B	6.32×	5.99×	6.05×	5.56×
OPT-6.7B	7.32×	6.84×	6.08×	4.93×
OPT-13B	7.37×	6.46×	5.77×	5.05×
OPT-30B	5.49×	5.07×	4.89×	4.45×
BLOOM-560M	7.37×	6.95×	6.82×	6.30×
BLOOM-1.1B	6.37×	6.21×	6.19×	6.04×
BLOOM-3B	5.41×	5.35×	5.33×	5.18×
BLOOM-7.1B	5.51×	5.54×	5.47×	5.35×

TABLE IX
END-TO-END DECODING SPEED-UP IN TERMS OF TPOT ON LMSYS-CHAT-1M INPUTS.

GPU	L3.2-1B	L3.1-8B	Q3.5-0.8B	Q3.5-9B
RTX 4090	1.11×	1.04×	1.04×	1.07×
DGX Spark	1.16×	1.06×	1.12×	1.09×
A100	1.01×	1.01×	1.00×	1.00×

Note: L3.2-1B, L3.1-8B, Q3.5-0.8B, and Q3.5-9B denote LLaMA3.2-1B, LLaMA3.1-8B, Qwen3.5-0.8B, and Qwen3.5-9B, respectively.

Table IX reports the end-to-end decoding latency measured in terms of TPOT in the actual generation pipeline using LMSYS-Chat-1M inputs. This experiment was conducted on LLaMA3.2-1B, LLaMA3.1-8B, Qwen3.5-0.8B, and Qwen3.5-9B. The input sequence length was fixed to 256, and 256 tokens were generated using greedy decoding. For RTX 4090, DGX Spark, and A100, the average value from more than 30 repeated runs after warm-up was used. The top-k, tile size, and cluster size of the proposed method were set to 256, 8, and 64, respectively.

In Table IX, the end-to-end speed-up differs depending on the GPU platform. RTX 4090 shows a TPOT speed-up of 1.04×–1.11× and DGX Spark a larger 1.06×–1.16×, showing that isolated LM Head acceleration also improves latency in the actual generation pipeline. In contrast, on A100, the speed-up remains at approximately 1.00× to 1.01×, since

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

TABLE X
LM HEAD ACCELERATION RATIO UNDER DIFFERENT BATCH SIZES.

Batch Size	L3.2-1B	L3.1-8B	Q3.5-0.8B	Q3.5-9B
1	5.26×	7.34×	5.48×	12.7×
2	4.62×	6.38×	4.57×	10.7×
4	4.11×	5.62×	4.33×	9.42×
8	3.91×	5.18×	3.69×	5.97×
16	2.82×	3.16×	2.38×	3.86×
32	1.83×	1.99×	1.70×	2.13×

Note: L3.2-1B, L3.1-8B, Q3.5-0.8B, and Q3.5-9B denote LLaMA3.2-1B, LLaMA3.1-8B, Qwen3.5-0.8B, and Qwen3.5-9B, respectively.

the original pipeline already utilizes high memory bandwidth and parallelism, and end-to-end latency is also governed by the Transformer block, KV cache access, memory hierarchy, and kernel launch overhead. Therefore, the proposed method can provide system-level TPOT improvement, particularly in latency-sensitive generation environments such as RTX 4090 and DGX Spark.

Table X reports the LM Head-only acceleration ratio measured as the batch size increases. This experiment used inputs randomly selected from ShareGPT, and the sequence length was fixed to 256. Measurements were conducted on DGX Spark for LLaMA3.2-1B, LLaMA3.1-8B, Qwen3.5-0.8B, and Qwen3.5-9B, and the average value over more than 30 repeated runs after warm-up was used. Token selection was set to greedy decoding, and the top-k, tile size, and cluster size of the proposed method were set to 256, 8, and 64, respectively.

In Table X, the speed-up gradually decreases as the batch size increases, yet all models maintain acceleration above $1\times$ even at a batch size of 32 ($1.70\times$–$2.13\times$). This trend follows from the computational form of the original LM Head: at small batch sizes, the original LM Head is implemented in a form close to token-wise GEMV, and large-vocabulary weight access becomes a major burden on latency and memory bandwidth, where the proposed candidate-reduction structure yields high speed-up. As the batch size increases, the original LM Head computation changes into a larger GEMM form, improving GPU parallelism and arithmetic utilization, so the relative benefit decreases; nevertheless, the batch-aware CUDA implementation of the proposed method retains acceleration even at a batch size of 32.

In summary, Tables VII–X show that the proposed method reduces the memory footprint for logits computation while limiting accuracy degradation, provides consistent LM Head-only speed-up across various LLM families and batch sizes, and yields practical end-to-end TPOT improvements on RTX 4090 and DGX Spark. The limited gains on A100 and at large batch sizes are not a fundamental limitation of the proposed structure, but indicate the need for architecture-aware CUDA kernel optimization that fully exploits HBM bandwidth, cache hierarchy, top-k selection, LUT accumulation, and candidate-wise fine projection; further system-level improvement can thus be expected on bandwidth-rich GPUs, considering recent architectures such as Hopper or Blackwell.

TABLE XI
HIT RATIO AND COMPRESSION RATIO OBTAINED BY SWEEPING THE TILE SIZE (CLUSTER SIZE FIXED TO 64) AND THE CLUSTER SIZE (TILE SIZE FIXED TO 4).

Size	Tile Sweep		Cluster Sweep	
	Hit Ratio	Comp. Ratio	Hit Ratio	Comp. Ratio
4	100.0%	9.425%	100.0%	3.128%
8	100.0%	4.737%	100.0%	4.694%
16	99.22%	2.394%	100.0%	6.262%
32	88.28%	1.222%	100.0%	7.837%
64	59.38%	0.636%	100.0%	9.425%
128	31.64%	0.343%	100.0%	11.04%
256	28.12%	0.196%	100.0%	12.70%
512	29.30%	0.123%	100.0%	14.46%

Note: Comp. Ratio denotes the compression ratio relative to the original LM Head weights. In the Tile Sweep, Size denotes the tile size with the cluster size fixed to 64; in the Cluster Sweep, Size denotes the cluster size with the tile size fixed to 4.

C. Ablation Study

To analyze how the hyperparameters of the proposed method, such as tile size, cluster size, and top-k, affect top-1 preservation performance, storage cost, and speed-up, we conducted three ablation studies. First, we fixed the cluster size and varied the tile size applied to the LM Head weights. Through this experiment, we observed how tiling influences the top-1 hit ratio and the resulting compression ratio. Second, we fixed the tile size and varied the cluster size. This experiment examines how the growth in codebook size and the increase in index map bit width affect the compression ratio. It also verifies whether increasing the cluster size improves the hit ratio while keeping the candidate set size constant. Finally, we fixed the tile size and cluster size and varied top-k to analyze how the hit ratio, LM Head-only speed-up, and end-to-end speed-up change. In the first two experiments, top-k was fixed at 256. We compared the hit ratio, defined as the fraction of decoding steps in which the original top-1 token is included in the coarse-stage candidate set, and the compression ratio relative to the original weights, including the codebook and index map. We used the LLaMA3.2-1B-Instruct model with the prompt "How to learn LLM?" and generated 256 tokens via greedy decoding to compute the hit ratio and speed-up. The results are summarized in Tables XI and XII.

Table XI reports the results of sweeping the tile size from 4 to 512 with the cluster size fixed to 64 (Tile Sweep), and of sweeping the cluster size over the same range with the tile size fixed to 4 (Cluster Sweep). In the Tile Sweep, as the tile size increases, the compression ratio decreases from 9.425% to 0.123%, since a larger tile size reduces the number of tiles, thereby decreasing the storage required for the index map. In contrast, the hit ratio remains at 100% for tile sizes 4 and 8, but it drops to 99.22% at 16, then decreases sharply to 88.28% at 32, and to 59.38% at 64. This occurs because increasing the tile size increases the dimensionality of the vectors clustered within each tile, while the cluster size is fixed. As a result, the codebook cannot sufficiently represent the higher-dimensional space, and the approximation error grows. Consequently, the original top-1 token is omitted more

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

TABLE XII
HIT RATIO AND RUNTIME SPEED-UP OBTAINED BY SWEEPING TOP-k.

Top-k	Hit Ratio	Speed-up	
		LM Head Only	End-to-end
2	74.41%	3.93×	1.11×
4	84.38%	3.93×	1.11×
8	91.80%	3.92×	1.11×
16	94.92%	3.92×	1.11×
32	96.48%	3.92×	1.11×
64	98.44%	3.88×	1.11×
128	99.61%	3.89×	1.11×
256	100.0%	3.86×	1.11×
512	100.0%	3.84×	1.11×
1,024	100.0%	3.75×	1.11×
2,048	100.0%	3.69×	1.11×
4,096	100.0%	3.53×	1.10×
8,192	100.0%	3.20×	1.09×

frequently from the coarse-stage candidate set when $k = 256$.

In the Cluster Sweep of Table XI, the hit ratio remains 100% even when the cluster size is reduced to 4, indicating that, for a tile size of 4, the tile dimension is sufficiently small, so a small codebook is sufficient to stably include the top-1 token in the coarse stage. In contrast, the compression ratio increases from 3.128% to 14.46% as cluster size increases. This is because the codebook storage grows linearly with the cluster size C, and the index map storage also increases because the packed index representation requires a bit width of $\lceil \log_2 C \rceil$. Therefore, for the same candidate set size ($k = 256$), increasing the cluster size does not improve the hit ratio and only increases storage cost.

Table XII shows the results obtained by fixing the tile size and cluster size to 4 and 4, respectively, and sweeping top-k from 2 to 8,192. As top-k increases, the hit ratio improves, and when a sufficiently large candidate set is secured, the original top-1 token is included in the coarse stage. In contrast, as top-k becomes larger, the number of candidate weight vectors processed in the fine stage increases, and therefore, the LM Head-only speed-up gradually decreases. The end-to-end speed-up changes only slightly with top-k because the effects of the Transformer block, KV cache access, kernel launch overhead, and other factors are combined. In addition, the batch size 1 result of LLaMA3.2-1B in Table X, obtained with a tile size of 8, shows a higher LM Head-only speed-up of 5.26× than the 3.86× obtained here with a tile size of 4. This is because, as tile size decreases, the number of tiles increases, which increases the coarse-stage LUT lookup and accumulation overhead. Therefore, in runtime optimization, the effect of tile size is more dominant than that of top-k, and this setting indicates that a tile size of 8 is more beneficial than 4.

In summary, tile size, cluster size, and top-k directly affect storage cost, top-1 preservation, and runtime, respectively. Increasing the tile size improves the compression ratio, but may reduce the hit ratio. When the tile size is sufficiently small, increasing the cluster size only increases the storage cost without further improving the hit ratio. In addition, increasing top-k improves the hit ratio, but increases fine-stage computation and therefore reduces the LM Head-only speed-up. Therefore, the hyperparameters of the proposed method should be selected by jointly considering storage cost and runtime, within a range that ensures a 100% hit ratio.

V. CONCLUSION

This paper proposed a coarse-fine LM Head algorithm to accelerate the full-vocabulary projection of the LM Head, which is repeatedly performed during autoregressive decoding. The proposed method partitions the LM Head weights into tiles along the hidden dimension and constructs a compact codebook and an index map offline by applying k-means clustering to the vocabulary-wise weight vectors of each tile. During online decoding, full-vocabulary coarse scores are computed using activation-dependent LUTs and index streaming, and exact GEMV or GEMM using the original LM Head weights is performed only for the selected top-k candidates. This reduces full-precision weight access and multiplication cost while preserving candidate-level exact computation. Evaluation results show that the proposed method limits the average accuracy degradation to 0.21%p compared with the original LM Head across nine downstream benchmarks, while accelerating the LM Head-only runtime by 5.82× on average and up to 12.7×. In addition, it achieves an end-to-end TPOT speed-up of up to 1.18× on DGX Spark, with consistent performance preservation and acceleration across various model families, decoding lengths, sampling conditions, and batch sizes. These results show that the proposed coarse-fine LM Head can serve as a practical LM Head acceleration method across various LLM inference environments.

ACKNOWLEDGMENT

This research was supported by Culture, Sports and Tourism R&D Program through the Korea Creative Content Agency grant funded by the Ministry of Culture, Sports and Tourism in 2025 (Project Name: Development of Digital Twin and Simulation Technology for Optimization of Safety Management in Mobile Performance Hall, Project Number: RS-2025-02221168, Contribution Rate: 100%). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[2] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
[3] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.
[4] Gemma Team, M. Riviere, S. Pathak *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.
[5] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[6] A. Yang, B. Yang, B. Zhang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
[7] K. Bae, E. Choi, K. Choi *et al.*, "EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes," *arXiv preprint arXiv:2507.11407*, 2025.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429

[8] E. Choi, K. Choi, S. Hong, J. Hwang, H. Jeon, H. Jo *et al.*, "K-EXAONE technical report," *arXiv preprint arXiv:2601.01739*, 2026.

[9] K. Shim, M. Lee, I. Choi, Y. Boo, and W. Sung, "SVD-softmax: Fast softmax approximation on large vocabulary neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5463–5473.

[10] P. H. Chen, S. Si, S. Kumar, Y. Li, and C.-J. Hsieh, "Learning to screen for fast softmax inference on large vocabulary neural networks," in *International Conference on Learning Representations*, 2019.

[11] W. Tranheden, S. Ahmed, D. Dubhashi, J. Matthiesen, and H. von Essen, "FlashHead: Efficient drop-in replacement for the classification head in language model inference," *arXiv preprint arXiv:2603.14591*, 2026.

[12] R. Goel, S. Agrawal, M. Gagrani, J. Park, Y. Zao, H. Zhang, T. Liu, Y. Yang, X. Yuan, J. Lu, C. Lott, and M. Lee, "VOCABTRIM: Vocabulary pruning for efficient speculative decoding in LLMs," *arXiv preprint arXiv:2506.22694*, 2025.

[13] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 19 274–19 286.

[14] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[15] W. Zhao, T. Pan, X. Han, Y. Zhang, A. Sun, Y. Huang, K. Zhang, W. Zhao, Y. Li, J. Zhou, H. Zhou, J. Wang, Z. Liu, and M. Sun, "FR-spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, Vienna, Austria, 2025, pp. 3909–3921.

[16] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, Melbourne, Australia, 2018, pp. 889–898.

[17] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *International Conference on Learning Representations*, 2020.

[18] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[19] M. Renze, "The effect of sampling temperature on problem solving in large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, Florida, USA, 2024, pp. 7346–7356.

[20] J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen, and S. Mehrotra, "Draft & verify: Lossless large language model acceleration via self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 11 263–11 282. [Online]. Available: https://aclanthology.org/2024.acl-long.607/

[21] H. Xia, Y. Li, J. Zhang, C. Du, and W. Li, "SWIFT: On-the-fly self-speculative decoding for LLM inference acceleration," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=EKJhH5D5wA

[22] M. Song, H. Xia, J. Zhang, C. T. Leong, Q. Xu, W. Li, and S. Li, "KNN-SSD: Enabling dynamic self-speculative decoding via nearest neighbor layer set optimization," in *Findings of the Association for Computational Linguistics: EACL 2026*. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 641–655. [Online]. Available: https://aclanthology.org/2026.findings-eacl.31/

[23] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html

[24] T. Liu, Y. Shen, X. Hu, B. Zhang, H. Zhang, J. Dai, J. Zhang, S. Ge, L. Chen, Y. Li, and M. Wan, "When hidden states drift: Can KV caches rescue long-range speculative decoding?" 2026. [Online]. Available: https://arxiv.org/abs/2604.26412

[25] Y. Shen, T. Liu, X. Hu, Q. Kong, B. Zhang, J. Dai, J. Zhang, S. Ge, L. Chen, Y. Li, M. Wan, and C. Wang, "Draft less, retrieve more: Hybrid tree construction for speculative decoding," 2026. [Online]. Available: https://arxiv.org/abs/2605.20104

[26] J. Zhang, Y. Ji, F. Ren, Y. Li, B. Zeng, Z. Chen, K. Chen, L. Shou, G. Chen, and H. Li, "Efficient inference for large vision-language models: Bottlenecks, techniques, and prospects," in *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 21 036–21 066. [Online]. Available: https://aclanthology.org/2026.findings-acl.1057/

[27] Y. Ji, J. Zhang, H. Xia, J. Chen, L. Shou, G. Chen, and H. Li, "SpecVLM: Enhancing speculative decoding of video LLMs via verifier-guided token pruning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 7205–7219. [Online]. Available: https://aclanthology.org/2025.emnlp-main.366/

[28] Y. Ji, J. Zhang, J. Chen, C. Wang, L. Shou, G. Chen, and H. Li, "See the forest for the trees: Loosely speculative decoding via visual-semantic guidance for efficient inference of video LLMs," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 23 707–23 726. [Online]. Available: https://aclanthology.org/2026.acl-long.1087/

[29] RyokoAI, "ShareGPT52K: A dataset of user-shared chatgpt conversations," Hugging Face dataset, 2023.

[30] L. Zheng, W.-L. Chiang, Y. Sheng, T. Li, S. Zhuang, Z. Wu, Y. Zhuang, Z. Li, Z. Lin, E. P. Xing, J. E. Gonzalez, I. Stoica, and H. Zhang, "LMSYS-chat-1m: A large-scale real-world LLM conversation dataset," in *International Conference on Learning Representations*, 2024.

[31] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, Bangkok, Thailand, 2024, pp. 3119–3137.

[32] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations*, 2021.

[33] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try ARC, the AI2 reasoning challenge," *arXiv preprint arXiv:1803.05457*, 2018.

[34] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy, 2019, pp. 4791–4800.

[35] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[36] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following evaluation for large language models," *arXiv preprint arXiv:2311.07911*, 2023.

[37] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, Dublin, Ireland, 2022, pp. 3214–3252.

[38] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about physical commonsense in natural language," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.

[39] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "WinoGrande: An adversarial winograd schema challenge at scale," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 8732–8740.

[40] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei, "Language models are multilingual chain-of-thought reasoners," in *International Conference on Learning Representations*, 2023.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Computers. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TC.2026.3725429



Jung-Woo Kim received the B.S. and M.S. degrees in Electronic Materials Engineering from Kwangwoon University, Seoul, Korea, in 2024 and 2026, respectively. His research interests include deep neural networks, LLM inference acceleration, neural processing units, and SoC design.



Seung-Hwan Yoon received the B.S. degree in Electronic Materials Engineering from Kwangwoon University, Seoul, Korea, in 2025. He is currently pursuing the M.S. degree in Electronic Materials Engineering at Kwangwoon University. His research interests include FPGA-based acceleration, deep learning systems, and video compression algorithms.



Hye-Rin Lee is currently pursuing the B.S. degree in Electronic Convergence Engineering at Kwangwoon University, Seoul, Korea. Her research interests include FPGA-based acceleration, deep learning systems, and video compression algorithms.



Hak-Bum Lee received the B.S. degree in Electronic Materials Engineering from Kwangwoon University, Seoul, Korea, in 2024. He is currently pursuing the M.S. degree in Electronic Materials Engineering at Kwangwoon University. His research interests include multi-camera-based 3D vision, deep learning, and quantization techniques for deploying deep neural networks on FPGA platforms.



Su-Min Oh received the B.S. degree in Electronic Materials Engineering from Kwangwoon University, Seoul, Korea, in 2025. She is currently pursuing the M.S. degree in Electronic Materials Engineering at Kwangwoon University. Her research interests include deep learning model-based hardware, NPU design, and video compression algorithms.



Seung-Yoon Kang is currently pursuing the B.S. degree in Electronic Engineering at Kwangwoon University, Seoul, Korea. His research interests include FPGA and ASIC design, AI acceleration, and inference optimization.



Young-Ho Seo received the M.S. and Ph.D. degrees from the Department of Electronic Materials Engineering, Kwangwoon University, Seoul, Korea, in 2000 and 2004, respectively. He was a visiting professor at the University of Nebraska at Omaha, USA. He is currently a full professor in the Department of Electronic Materials Engineering at Kwangwoon University, a director of the Realistic Media Research Center at Kwangwoon University, and a Chief Technical Officer as a co-founder at Omotion Inc. His research interests include 2D and 3D image processing, real-time systems, hardware-based deep neural networks, and parallel processing.

Authorized licensed use limited to: CITY UNIV OF HONG KONG. Downloaded on September 17,2026 at 21:00:04 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.